

Aula 6 – Encapsulamento e Métodos Decoradores (POO III)

Aprenda a proteger e controlar os dados das suas classes de forma elegante e profissional

Introdução

O que já aprendemos

- Classes, objetos e atributos
- Métodos e comportamentos
- Herança e polimorfismo

Estes conceitos são fundamentais para **organizar melhor o código** e garantir **segurança e controle** sobre os dados das suas aplicações.

Novos conceitos

- **Encapsulamento** – proteção de dados
- **Métodos decoradores** – property, setter e deleter



O que é Encapsulamento



Proteção de Dados

Impede acesso direto aos atributos sensíveis da classe



Controle de Acesso

Define quem pode ver ou modificar os dados internos



Convenções Python

Usa `_`, `__` antes do nome para indicar nível de proteção

Níveis de Visibilidade em Python

1

atributo

Público – acesso livre

2

_atributo

Protegido – uso interno

3

__atributo

Privado – totalmente protegido

Exemplo Prático

```
class ContaBancaria:
    def __init__(self, titular, saldo):
        self.titular = titular
        self.__saldo = saldo # atributo privado

    def mostrar_saldo(self):
        print(f"Saldo de {self.titular}: R${self.__saldo}")

    def depositar(self, valor):
        self.__saldo += valor
        print("Depósito realizado com sucesso!")

conta = ContaBancaria("Carlos", 1000)
conta.mostrar_saldo()
conta.depositar(500)
```

📌 O saldo é protegido — só pode ser alterado por métodos da própria classe, garantindo segurança e validação.

Exercício 1 – Praticando Encapsulamento

1

Criar a Classe

Defina uma classe `Produto` com atributos privados `__nome` e `__preco`

2

Método de Exibição

Implemente `mostrar_detalhes()` que exiba nome e preço do produto

3

Aplicar Desconto

Crie `aplicar_desconto(percentual)` para reduzir o preço de forma controlada

Desafio

Teste o código criando um produto e aplicando um desconto de 10%. Observe como os atributos privados mantêm os dados protegidos!



Métodos Getters e Setters



get_atributo()

Método para **obter** o valor de um atributo privado

set_atributo(valor)

Método para **modificar** o valor com validação e controle

Por que usar?

Getters e setters são a **ponte segura** para acessar atributos privados.

Exemplo na Prática

```
class Pessoa:
    def __init__(self, nome):
        self.__nome = nome

    def get_nome(self):
        return self.__nome

    def set_nome(self, novo_nome):
        self.__nome = novo_nome

p1 = Pessoa("Maria")
print(p1.get_nome()) # Maria
p1.set_nome("Ana")
print(p1.get_nome()) # Ana
```

Com **getters e setters**, você mantém o controle total sobre como os dados são acessados e modificados, adicionando validações sempre que necessário.

Evoluindo com @property

Python oferece uma forma mais **elegante e pythônica** de trabalhar com getters e setters: o decorador **@property**.



Código mais Limpo

Acesse métodos como se fossem atributos, sem parênteses



Mais Seguro

Mantém o encapsulamento enquanto simplifica o acesso



Mais Pythônico

Segue as melhores práticas da linguagem Python

Exemplo com @property

```
class Retangulo:
    def __init__(self, largura, altura):
        self.__largura = largura
        self.__altura = altura

    @property
    def area(self):
        return self.__largura * self.__altura

r = Retangulo(5, 3)
print(r.area) # 15 - sem precisar chamar como método!
```



Observe como `r.area` parece um atributo comum, mas na verdade executa um método protegido!

Métodos Decoradores em Python

Decoradores são **funções especiais** que modificam o comportamento de métodos e atributos, tornando seu código mais poderoso e expressivo.

@property

Transforma um método em atributo de **leitura** – permite obter o valor

@nome.setter

Permite **alterar** o valor com validação e regras de negócio

@nome.deleter

Controla a **exclusão** do atributo de forma segura

Benefício principal: aplicar regras de validação e controle sem alterar a forma como o código é acessado externamente.



Exemplo Completo de Decoradores

```
class Pessoa:
    def __init__(self, nome):
        self.__nome = nome

    @property
    def nome(self):
        return self.__nome

    @nome.setter
    def nome(self, novo_nome):
        if len(novo_nome) < 3:
            print("Nome muito curto!")
        else:
            self.__nome = novo_nome

    @nome.deleter
    def nome(self):
        print("Nome apagado!")
        del self.__nome

p = Pessoa("João")
print(p.nome)    # João
p.nome = "Lu"    # Nome muito curto!
p.nome = "Lucas" # OK, altera para Lucas
del p.nome       # Nome apagado!
```



Validação Automática

O **setter** verifica se o nome tem pelo menos 3 caracteres



Controle Total

Decide exatamente como os dados podem ser modificados



Proteção de Dados

Garante que apenas valores válidos sejam armazenados



Exercício 2 – Dominando Decoradores



Criar Classe Aluno

Defina atributo privado `__nota`



Property para Leitura

Use `@property` para retornar a nota



Setter com Validação

Use `@nota.setter` para aceitar apenas valores entre 0 e 10



Testar o Código

Teste com notas válidas e inválidas



Desafio Bônus

Crie um método `situacao()` que retorne "Aprovado" se a nota for maior ou igual a 7, e "Reprovado" caso contrário. Isso reforça seu domínio sobre métodos em classes!



Conclusão



Encapsulamento

Protege e organiza melhor os dados, evitando modificações indevidas



Getters e Setters

Controlam o acesso aos atributos com validação e segurança



Propriedades e Decoradores

Tornam o código mais elegante, seguro e pythônico

Próximo Passo

Com estes conceitos, você deu um passo importante rumo a uma **POO mais robusta e profissional**. Continue praticando e explorando as possibilidades!

Parabéns por completar esta aula! Agora você está preparado para criar classes mais seguras, organizadas e seguindo as melhores práticas do Python.